The Raspberry Pi FM Transmitter (Pi-FM)

Deep Dive: PLLC, DMA, and Signal Generation

Aditya Appana and Suraj N

February 16, 2026

Status Report

- System Initialization Setup OS Deployment: The Raspberry Pi was successfully initialized by flashing the operating system image onto the SD card and booting the device.
  Installation: We utilized the pifm-basic software package to turn the Raspberry Pi's GPIO pins into a makeshift FM transmitter. This software allows for Frequency Modulation (FM) directly from the digital pins without requiring an external radio add-on board.

Status Report

2. Transmission Testing
   - Initial Broadcast: To verify the setup, we broadcasted standard test files (sound.wav and stereo$_4$4100.*wav*)*usingthesoftware*.

- Reception: We tuned a standard FM radio receiver to the target frequency and successfully demodulated the signal, confirming clear audio reception from the Pi.
- Variable Testing: We proceeded to test the system's versatility by transmitting a variety of different audio files across multiple frequencies to ensure stability across the FM band.

Status Report

3. Performance Challenges Troubleshooting During the stress-testing phase, we encountered significant signal degradation characterized by the following:

- Symptoms: The audio would transmit clearly for only a few seconds before deteriorating into static bursts or complete noise.
- Root Cause Analysis: We identified two primary factors causing the instability:
- Undervoltage: The input voltage was fluctuating below the required threshold.

Status Report

- Thermal Throttling: The CPU began to overheat during prolonged transmission.
- Technical Note: Because pifm relies on precise CPU timing to generate radio waves (bit-banging), any throttling (reduction in CPU clock speed) immediately corrupts the carrier frequency, resulting in noise. Resolution: The issue was rectified by stabilizing the power supply to maintain consistent voltage, which prevented the CPU from throttling and restored the integrity of the transmission.

Status Report

4. Final Observations Range Test Effective Range: Under stable power conditions, we measured a clear transmission range of approximately 20 meters using a standard wire antenna connected to the GPIO pin.

What is Pi-FM?

- The Raspberry Pi FM project (PiFM/rpitx) is a software-based hack that transforms a Pi into an FM radio transmitter.
- **Key Characteristic:** It requires **no additional radio hardware**.
- **Mechanism:** It generates radio waves purely through software manipulation of the Pi's digital pins.
- **Stereo Support:** It can generate a Multiplex (MPX) signal including Left/Right audio and a 19 kHz pilot tone.
- **RDS Support:** It can broadcast station names and scrolling text via a 57 kHz subcarrier.

The Core Engine: PLLC

- **Definition:** In the context of the Raspberry Pi Broadcom processor, **PLLC** stands for *Phase Locked Loop C*.
- **Function:** It is an internal hardware circuit used to generate clock frequencies for the system.
- **Standard Role:**
  - Drives Core Logic (VPU and 3D GPU).
  - Derives clock speed for SDRAM.
- **Role in Pi-FM:** PLLC acts as the **carrier frequency generator**.
- **Why PLLC?** It allows for high frequency resolution, which is necessary to create clean audio without robotic distortion.

The Mechanism: "Bit Banging" a Frequency

- **Concept:** The Pi manipulates the internal clock generator effectively "hijacking" it to oscillate at radio frequencies (e.g., 100 MHz).
- **Signal Source:** Software like `rpitx` takes control of the PLLC.
- **Frequency Modulation (FM):**
  - The software rapidly changes the frequency thousands of times per second.
  - To transmit audio, the frequency speeds up and slows down in rhythm with the sound wave.
- **Output:** The oscillating signal is routed to **GPIO 4** (GPCLK0).

The Challenge: CPU Jitter

- **The Task:** Transmitting FM requires changing the pin frequency hundreds of thousands of times per second.
- **The Problem:** The CPU is busy running Linux (Wi-Fi, logs, SD card management).
- **The Consequence:** If the CPU is distracted for a microsecond, the frequency updates pause.
- **Result:** "Jitter"—audible pops, clicks, or silence on the radio.

The Solution: DMA (Direct Memory Access)

- **Role:** The DMA acts as a high-speed, automated courier service.
- **Mechanism:**
  - **CPU:** Decodes MP3s into a list of control block instructions stored in RAM.
  - **DMA:** Reads instructions from RAM and writes values directly to the Clock Generator registers.
- **Benefit:** Frequency changes happen perfectly on time, even if the OS freezes.
- Without DMA, the Pi would produce static or glitchy audio.

Pacing the Stream: The PWM Controller

- **The Issue:** The DMA is too fast; it would process a 3-minute song in a fraction of a second.
- **The Solution:** The PWM (Pulse Width Modulation) controller acts as a **metronome**.
- **Workflow:**
  - PWM "ticks" at the audio sample rate (e.g., 22,050 Hz).
  - DMA waits for a "Data Request" (DREQ) from the PWM before processing the next instruction.
- **Result:** Radio waves are modulated at the exact sample rate required for clear audio.

Component Comparison

Feature	Crystal Oscillator	PWM Controller
Type	Physical Hardware (Quartz)	Digital Logic (Software)
Frequency	Fixed (e.g., 19.2 MHz)	Variable (Programmable)
Role	Creates Carrier Wave Source	Controls Audio Speed/Pacing
Analogy	The Electricity	The Drummer

The Clock Generator (Frequency Synthesizer)

- **Role:** Takes the fast internal system clock (approx 500 MHz) and divides it down to the target FM frequency.
- **Fractional Division:**
  - Integer division is insufficient ($500/6 = 83.3$ MHz, gap is too large).
  - Uses **MASH noise shaping** to divide by decimals (e.g., 5.0001).
- **Modulation in Action:**
  - **High Voltage Audio:** Speed up to 100.075 MHz.
  - **Low Voltage Audio:** Slow down to 99.925 MHz.

Required Hardware Setup

- **Device:** Any Raspberry Pi model (Zero, 1, 2, 3, 4, 5).
- **Antenna:** A jumper wire measuring roughly **75cm (29.5 inches)**.
- **Connection:** Connect the wire to **GPIO Pin 4**.
- **Physics:** 75cm is roughly 1/4 the wavelength of FM radio waves, making it efficient.
- **Ground:** A wire connected to a GND pin improves stability.

Critical Warnings: Harmonics

Warning: Square Wave Harmonics

The GPIO pin outputs a **square wave**, not a smooth sine wave.

- **The Physics:** A pure 100 MHz sine wave radiates energy *only* at 100 MHz.
- **The Reality:** A 100 MHz square wave radiates energy at **100 MHz, 300 MHz, 500 MHz, 700 MHz** (odd harmonics).
- **The Danger:** You may accidentally broadcast on frequencies used by air traffic control, police, or mobile networks.
- **System Instability:** Manipulating PLLC can cause system crashes or SD card corruption, as it also drives the GPU/RAM.

Mitigating Risks: Band Pass Filter

- **Requirement:** To make the project safer and cleaner, responsible users add a **Band Pass Filter**.
- **Function:** A small circuit board connected between the Pi and the antenna.
- **Result:** Physically blocks harmonic frequencies (300 MHz, 500 MHz) while allowing the intended 100 MHz signal to pass.

Figure: Radio

Figure: Radio